DESIGN DOCUMENT

Amateur Radio Licence Exam Scheduling and Result Engine

Project ID: E036-ECE-061
Team: E036
Team Members: Shivaraj Patil and Sindhu
Technology: Python 3.10+

System Design and Implementation Overview

1. Purpose

The application processes amateur-radio examination applications, allocates eligible applicants to future examination sessions, grades section scores, applies exemptions and cooling periods, and produces auditable CSV reports.

It is a local, batch-oriented Python application controlled by a pipe-delimited command file. It does not require a database or network connection.

2. System Inputs and Outputs

Inputs are applicants.csv, past_results.csv, sessions.csv, scores.csv and commands.txt. The command file controls loading, application batches, seating, grading and reattempt reporting.

Output	Purpose
apply_log.csv	One decision row for each APPLY command.
seating.csv	One row per allocated seat and seating print command.
results.csv	One result row per allotted applicant per graded session.
reattempt.csv	Latest failed grade, failure streak, cooling days and earliest eligible date.

3. Main Components

- CSV input layer — reads and validates source records.
- Command processor — executes commands in file order and reports line-specific errors.
- Eligibility validator — checks age, requested grade and existing future allotments.
- Batch allotment engine — processes normal-priority applications before LOW-priority applications.
- Result engine — calculates section results, exemptions and overall outcomes.
- Reattempt engine — calculates failure streaks and cooling dates.
- CSV output layer — writes the four reports in fixed column order.

4. Processing Flow

- Load applicants, past results, sessions and score sheets.
- BATCH_START records the batch date; APPLY validates and queues eligible applications.
- BATCH_END processes normal-priority applications in command order, followed by LOW-priority applications in command order.
- Each application receives the earliest future matching-grade session satisfying age, cooling and seat-capacity rules.
- PRINT_SEATING records allocated seats; GRADE evaluates allotted applicants.
- PRINT_REATTEMPT creates a report using only results on or before its as-of date.
- The engine writes apply_log.csv, seating.csv, results.csv and reattempt.csv.

5. Business Rules

- Minimum age is 12 on the examination date.
- GENERAL eligibility requires a previous RESTRICTED overall pass or at least one year of prior licence experience.
- RESTRICTED THEORY and RULES each require 50%; GENERAL THEORY and RULES each require 60%; GENERAL MORSE requires 5 WPM.
- Each section is assessed independently. Overall PASS requires every required section to be PASS or EXEMPT.
- A same-grade section pass is carried through the inclusive 12-calendar-month window. EXEMPT does not renew the window.

- Cooling is 30 days after failure, or 90 days after three or more consecutive failures in the same grade. ABSENT does not count as a failure and does not break the streak.
- A sitting with no score rows is ABSENT. Two or more absences result in LOW priority.
- Percentages use Decimal arithmetic and ROUND_HALF_UP.

6. Data Structures and Algorithms

Dictionaries keyed by applicant ID and session ID provide direct record lookup. Past and current results are grouped by applicant; scores are grouped by (session_id, applicant_id) and section; allotments are grouped by session.

Batch allocation uses a stable two-pass ordering: normal priority first, then LOW priority. Each application scans matching sessions in examination-date order and selects the first session satisfying all constraints with a free seat.

Decimal arithmetic is used for percentages to avoid binary floating-point errors at pass-mark boundaries.

7. Validation and Error Handling

The program validates duplicate identifiers, dates, numeric seat counts, grades, score references, score sections, duplicate section scores and command arguments. Command errors include the physical line number and original command text.

Input filenames are resolved inside the configured data directory to prevent paths from escaping the input folder.

8. Testing Strategy

The pytest suite combines focused unit tests and a full sample-data integration test. It checks rounding, calendar-month boundaries, age, grade eligibility, seat priority, duplicate applications, cooling periods, exemption cut-offs, score thresholds, Morse WPM, unknown applicants, as-of filtering, row counts and exact output headers.

Run tests with: python -m pytest -q

9. Assumptions and Limitations

- The application is a local command-file processor, not a web application.
- The sample pack defines the 12-month exemption as a calendar-month boundary, inclusive of the matching date.
- The command file is authoritative for application processing order; timestamps are validated but do not reorder commands.
- For a partial score sheet, each required section is decided independently. A required section without a score or valid exemption is FAIL; zero score rows for the sitting is ABSENT.